

Web Application Penetration Test Report

Black-Box Security Assessment

`redacted-client-app.vercel.app`

2

CRITICAL FINDINGS

2

MED / LOW FINDINGS

4

CONTROLS VERIFIED

Prepared For

Client App (E-Commerce Platform)

Prepared By

Shadi Ibrahim

Assessment Date

June 18-19, 2026

Report Version

1.0

Table of Contents

Note on this Edition. This document is a portfolio/demonstration edition of a full penetration test report originally delivered to a client under a signed engagement. In accordance with the confidentiality clause of the engagement's written authorization, the client's real name, application name, and live URL have been replaced with generic placeholders throughout this document. All technical content — methodology, vulnerability detail, evidence, severity ratings, and recommendations — is otherwise presented exactly as it appeared in the original report. This edition is shared publicly with the client's knowledge and consent.

1. Executive Summary	03
2. Engagement Scope & Authorization	04
3. Methodology	05
4. Risk Rating Definitions	05
5. Findings Summary Table	06
6. Detailed Findings	07
6.1 Client-Side Price Manipulation	07
6.2 Broken Access Control — Admin Panel Exposure	09
6.3 Missing Rate Limiting on Authentication	12
6.4 Improper Error Handling (Unhandled Server Crash)	13
6.5 Verified Security Controls (Positive Findings)	14
7. Strategic Recommendations	16
8. Conclusion	17
Appendix A — Disclaimer & Confidentiality	18

1. Executive Summary

Between **June 18 and June 19, 2026**, a black-box web application penetration test was conducted against the Client App e-commerce platform (**`https://redacted-client-app.vercel.app`**), under explicit written authorization from the application owner. The objective of this engagement was to identify security weaknesses across authentication, authorization, business logic, and input validation controls prior to the platform's production launch.

The assessment identified **two critical-severity vulnerabilities** that pose immediate and material risk to the confidentiality of customer data and the financial integrity of the platform. A malicious authenticated user is currently able to (1) manipulate the price of any order to an arbitrary value — including near-zero amounts — and have that order fully confirmed for Cash-on-Delivery fulfilment, and (2) access the full administrative back-office panel without holding any administrative privilege, exposing the complete order history, customer phone numbers, and delivery addresses of every customer on the platform.

In addition, the assessment identified a **medium-severity** absence of rate limiting on the login endpoint, and a **low-severity** improper error handling issue that allows an unauthenticated crash condition to be triggered via malformed numeric input.

On a positive note, the assessment also confirmed that several critical defenses are correctly implemented: output encoding effectively neutralizes Stored Cross-Site Scripting (XSS) attempts, the session cookie's `SameSite=Lax` attribute mitigates Cross-Site Request Forgery (CSRF) on state-changing endpoints, and — notably — the administrative *write* operations (e.g. updating order status) are correctly protected server-side, even though the corresponding administrative *read* operations are not. This inconsistency is a recurring pattern observed throughout the assessment and is discussed further in Section 7.

Overall Risk Rating: CRITICAL. The combination of unrestricted administrative panel access and unauthenticated price tampering on Cash-on-Delivery orders represents an immediately exploitable path to financial loss and a large-scale exposure of customer Personally Identifiable Information (PII). Remediation of Findings 6.1 and 6.2 is strongly recommended before any public/production launch of the platform.

Risk Distribution



2. Engagement Scope & Authorization

This assessment was conducted under a signed written Penetration Testing Authorization Letter ("Get Out of Jail Free" letter) executed between the application owner (First Party / Client) and the Security Assessor (Second Party), explicitly granting permission to perform security testing against the asset listed below for the defined engagement window. A copy of the signed authorization is retained by the Assessor and is available upon request; it is not included in this technical report for confidentiality reasons.

Item	Detail
Target Application	https://redacted-client-app.vercel.app
Application Type	E-commerce web application (Next.js / React, hosted on Vercel, Supabase backend)
Testing Type	Black-Box, Authenticated & Unauthenticated Web Application Penetration Test
Methodology Basis	OWASP Top 10 (2025) & OWASP Web Security Testing Guide (WSTG)
Testing Window	June 18, 2026 – June 19, 2026
Authorization	Signed written consent obtained from the Client prior to testing

Out of Scope

- The underlying Vercel hosting infrastructure (platform-level testing was explicitly excluded; only the application layer owned by the Client was assessed)
- Denial of Service (DoS/DDoS) testing of any kind
- Social engineering or phishing campaigns against staff, employees, or customers
- Any modification, deletion, or exfiltration of real production data belonging to third-party customers
- Brute-force attacks carried out to completion (only proof-of-concept attempts were performed to confirm the absence of rate limiting; see Finding 6.3)

Testing Constraints. In accordance with the signed authorization, all testing was limited to non-destructive Proof-of-Concept (PoC) techniques. Where a vulnerability could plausibly be escalated to cause real damage (e.g. negative-value transactions, brute-force account takeover, or modification of real customer order data), the Assessor deliberately stopped at the point of confirmation and did not proceed to exploitation.

3. Methodology

Testing followed a structured manual methodology, supported by industry-standard tooling, across the following phases:

- **Reconnaissance:** Manual exploration of the application, API endpoint discovery via browser Developer Tools (Network tab), and traffic interception via Burp Suite Community Edition.
- **Authentication & Session Analysis:** Registration/login flow review, session cookie attribute inspection (HttpOnly, SameSite, Secure), and brute-force resilience testing.
- **Business Logic Testing:** Manipulation of client-controlled parameters (price, total) within checkout/order-creation requests using Burp Suite Repeater.
- **Authorization Testing (Broken Access Control):** Direct URL access attempts to privileged routes, role-boundary testing between standard user and administrative functions, and Insecure Direct Object Reference (IDOR) testing across order resources using a secondary test account.
- **Input Validation Testing:** Stored Cross-Site Scripting (XSS) payload injection and observation via DOM inspection; boundary/negative-value testing on numeric fields.
- **CSRF Analysis:** Inspection of state-changing request headers, cookie `SameSite` configuration, and Content-Type-based delivery feasibility analysis.

Tools Used

Tool	Purpose
Burp Suite Community Edition v2025.10.6	Traffic interception, request manipulation (Repeater)
Chrome DevTools	Network analysis, DOM/Elements inspection, Application/Cookie inspection
Manual Testing	All Proof-of-Concept payloads were manually crafted and verified

4. Risk Rating Definitions

Rating	Definition
CRITICAL	Directly exploitable, requires low skill/effort, results in severe business or data impact (e.g. financial loss, mass PII exposure, full account/data compromise).
MEDIUM	Exploitable under certain conditions; meaningfully increases risk of account compromise or abuse but is not immediately catastrophic alone.
LOW	Limited direct impact; typically a hardening or defense-in-depth issue rather than a directly exploitable breach.
PASS	Tested control was found to be properly implemented and functioning as intended; no remediation required.

5. Findings Summary

ID	Finding	Category	Severity
TH-01	Client-Side Price Manipulation on Order Creation	Business Logic / Improper Server-Side Validation	CRITICAL
TH-02	Broken Access Control — Unauthorized Admin Panel Read Access	Broken Access Control (OWASP A01:2025)	CRITICAL
TH-03	Missing Rate Limiting on Login Endpoint	Identification & Authentication Failures (OWASP A07:2025)	MEDIUM
TH-04	Improper Input Validation Leading to Server Crash (HTTP 500)	Mishandling of Exceptional Conditions (OWASP A10:2025)	LOW
TH-05	Stored Cross-Site Scripting (XSS) — Order Notes Field	Injection (OWASP A03:2025)	PASS
TH-06	Cross-Site Request Forgery (CSRF) on Order Creation	CSRF	PASS
TH-07	Administrative Write-Operation Authorization Check	Access Control	PASS
TH-08	IDOR Testing on Individual Order Retrieval Endpoint	Access Control	PASS / N/A

Note on Findings TH-02 and TH-07. These two findings, when read together, reveal an important and recurring pattern: the application correctly validates authorization on *write* (state-changing) requests server-side, but applies **no equivalent check on read requests**. This inconsistency is the direct cause of the Critical exposure described in TH-02, and is elaborated on in Section 7 (Strategic Recommendations).

6. Detailed Findings

TH-01

Client-Side Price Manipulation on Order Creation

CRITICAL

CWE: CWE-602 (Client-Side Enforcement of Server-Side Security)

OWASP: A06:2025 — Insecure Design

Endpoint: POST /api/orders

DESCRIPTION

The order-creation endpoint (`POST /api/orders`) accepts the `price` and `total` fields directly from the client-supplied JSON request body, and persists the order using these client-supplied values without re-validating them against the authoritative product price stored server-side. As a result, any authenticated user can set these fields to an arbitrary value of their choosing before submission.

- STEPS TO REPRODUCE**
1. Add the product "Classic Blue Jeans" (list price **\$79.99**) to the cart and proceed to checkout.
 2. Intercept the resulting `POST /api/orders` request using Burp Suite Proxy/Repeater.
 3. Modify the JSON body, changing `"price": 79.99` to `"price": 2`, and `"total": 79.99` to `"total": 2`.
 4. Forward the modified request to the server.
 5. Observe the server response: `HTTP/2 201 Created`, echoing back the manipulated price of `2` as the confirmed order value.

EVIDENCE — MANIPULATED REQUEST & SERVER RESPONSE (201 CREATED)

The screenshot displays the Burp Suite Repeater interface. The left pane shows the intercepted request, which has been modified to set `"price": 2` and `"total": 2` in the JSON body. The right pane shows the server's response, which is a `201 Created` status with a `Content-Type: application/json` header. The response body contains a JSON object with the manipulated values: `"price": 2` and `"total": 2`. The status bar at the bottom indicates the response is 855 bytes and took 2,040 milliseconds to receive.

Figure 1. Burp Repeater request with the tampered `price / total` fields set to `2` (originally \$79.99), and the server's `201 Created` response echoing back the manipulated order — confirmed with status `"pending"` and no price correction applied.

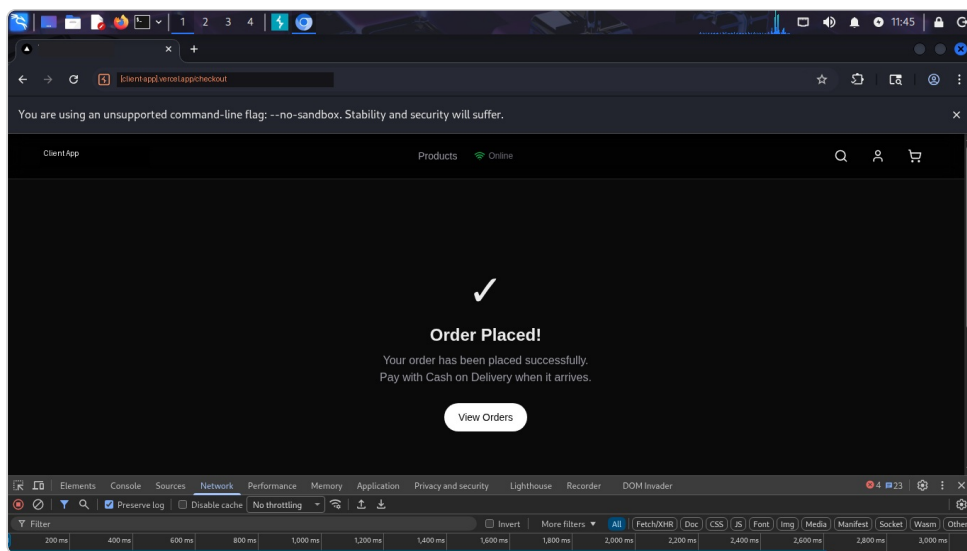


Figure 2. Front-end confirmation screen following the tampered request, confirming the order was accepted end-to-end with no downstream price re-validation step (Cash on Delivery flow).

IMPACT

Because Client App's sole payment method is **Cash on Delivery**, there is no third-party payment gateway (e.g. Stripe/PayPal) positioned to re-validate the transaction amount before fulfilment. An attacker can therefore purchase any item on the platform for an arbitrary price — including \$0.01 — and the order will be fully confirmed and queued for physical fulfilment at the manipulated price. This translates directly into uncapped financial loss for the merchant at scale.

RECOMMENDATION

- The server must independently compute `price` and `total` for every line item by looking up the authoritative price from the database using the submitted `product_id`. Client-supplied price/total values must be fully ignored on the server side.
- Apply the same server-side recalculation principle to any discount/coupon logic, should one be introduced in the future.
- Add automated regression tests that assert order totals always match the database-derived price, regardless of client input.

TH-02

Broken Access Control — Unauthorized Admin Panel Read Access

CRITICAL

CWE: CWE-862 (Missing Authorization)

OWASP: A01:2025 — Broken Access Control

Route: /admin/orders, /admin/products

DESCRIPTION

The administrative back-office panel, located at `/admin`, is reachable by simply navigating to the URL directly while authenticated as a **standard, non-privileged user account**. No server-side role check redirects or blocks the request. As a result, the panel renders fully, exposing the complete order history of every customer on the platform — including full names, delivery addresses, phone numbers, order contents, and timestamps — as well as full read/edit access to the product catalog.

The tested account's role was independently confirmed as `"role": "User"` via the account's own profile page prior to this test, ruling out any possibility of pre-existing administrative privilege.

STEPS TO REPRODUCE

1. Register and authenticate as a standard user (no administrative role assigned).
2. Confirm the account's role via `/dashboard` → Profile.
3. Navigate directly to `https://redacted-client-app.vercel.app/admin/orders`.
4. Observe that the Admin Panel renders without any access-denied response, redirect, or authentication challenge.
5. Repeat for `/admin/products` — full product catalog with edit/delete controls is similarly exposed.

EVIDENCE — CONFIRMED STANDARD USER ROLE

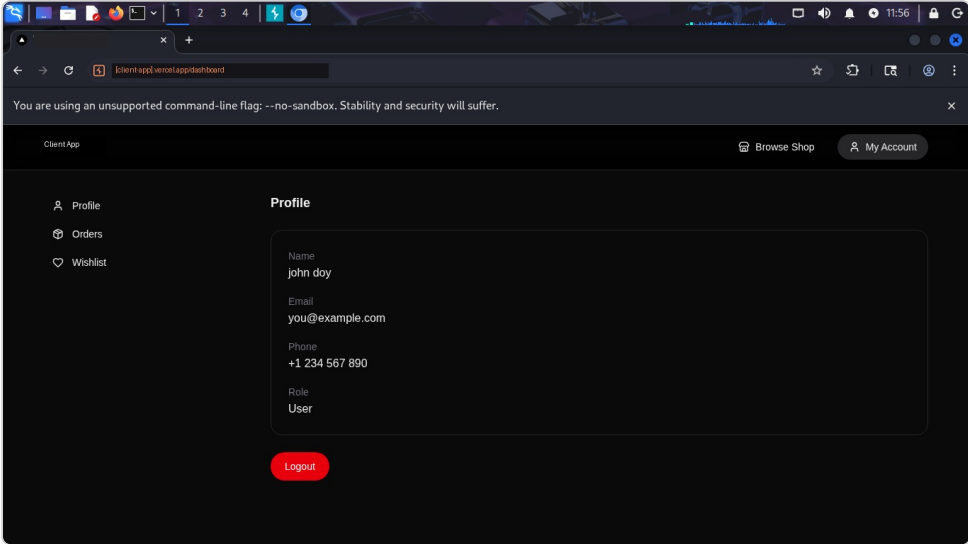


Figure 3. Account profile confirming "Role: User" — no administrative privilege assigned.

EVIDENCE — FULL ADMIN ORDERS PANEL (REACHABLE AS STANDARD USER)

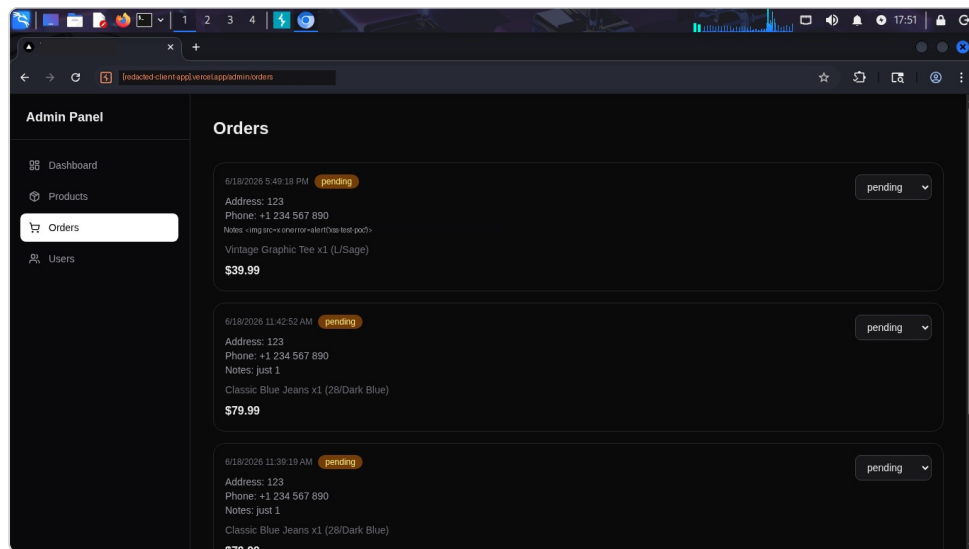


Figure 4. Admin Panel → Orders, accessed while authenticated as a standard user. Customer phone numbers, delivery addresses, and order details are fully visible. (Values shown are test/placeholder data entered by the Assessor.)

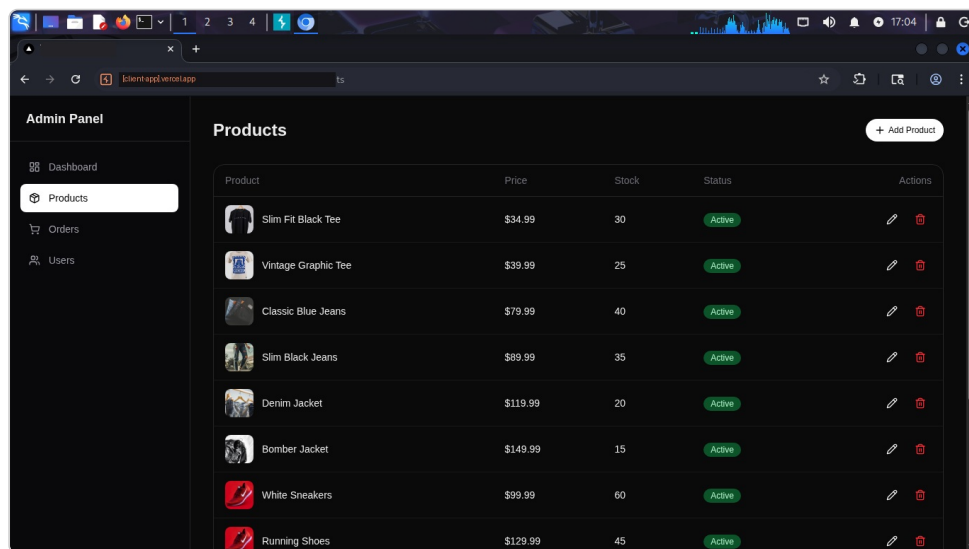


Figure 5. Admin Panel → Products, showing full product catalog with Edit/Delete controls exposed to a non-privileged account.

IMPACT

This finding constitutes a large-scale exposure of customer Personally Identifiable Information (PII) — names, phone numbers, and physical delivery addresses — to any registered user of the platform, regardless of privilege level. In addition, the exposed product catalog could allow an attacker to view internal stock levels and pricing data not normally intended for public/customer visibility. Combined with TH-01, this finding also means an attacker has full visibility into every fraudulent order's downstream status. This is classified as **Critical** under OWASP A01:2025 (Broken Access Control), as it requires no special tooling, no privilege escalation exploit, and is reachable by any registered user via direct URL navigation.

RECOMMENDATION

- Implement server-side route protection (e.g. Next.js middleware) that verifies the requesting user's role is `admin` before rendering any route or returning any data under `/admin/` or its corresponding API endpoints.
- Apply this check at the data-fetching layer (API/server actions), not only at the UI routing layer — a hidden button or client-side redirect is not a security control.
- Conduct a full audit of all `/admin/` routes and their corresponding API endpoints to ensure consistent enforcement, given that TH-07 confirms write operations are already correctly protected — the same protection pattern should be extended to all read operations.

SUPPLEMENTARY EVIDENCE — ORDER STATUS CONTROL EXPOSED IN UI

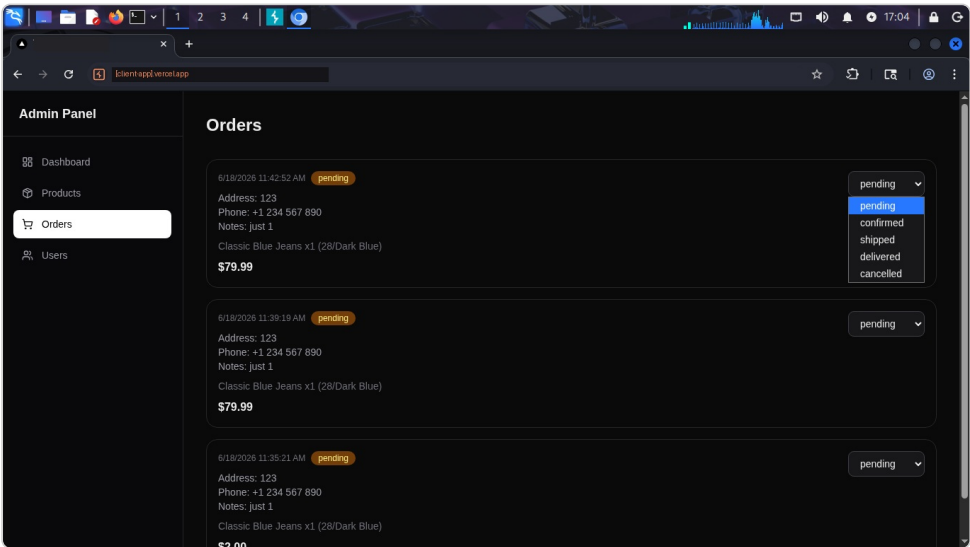


Figure 6. The order-status dropdown (Pending / Confirmed / Shipped / Delivered / Cancelled) is rendered and interactive in the UI for a non-privileged user, despite the underlying write operation being correctly blocked server-side (see Finding TH-07, page 14). This demonstrates the read/write enforcement inconsistency referenced in Section 5.

TH-03

Missing Rate Limiting on Login Endpoint

MEDIUM

CWE: CWE-307 (Improper Restriction of Excessive Authentication Attempts)**OWASP:** A07:2025 — Identification & Authentication Failures**Endpoint:** /login (POST /api/auth/...)

DESCRIPTION

The login form does not implement any account lockout, progressive delay, or CAPTCHA challenge after repeated failed authentication attempts. Seven consecutive failed login attempts using an incorrect password against the same account were submitted without triggering any throttling, lockout message, or additional verification step — each attempt returned an identical, immediate "Invalid login credentials" response.

STEPS TO REPRODUCE

1. Submit the login form with a valid email address and an incorrect password.
2. Repeat the submission 7 times in immediate succession.
3. Observe that the server returns an identical error response each time, with no lockout, delay, or CAPTCHA challenge introduced.

EVIDENCE

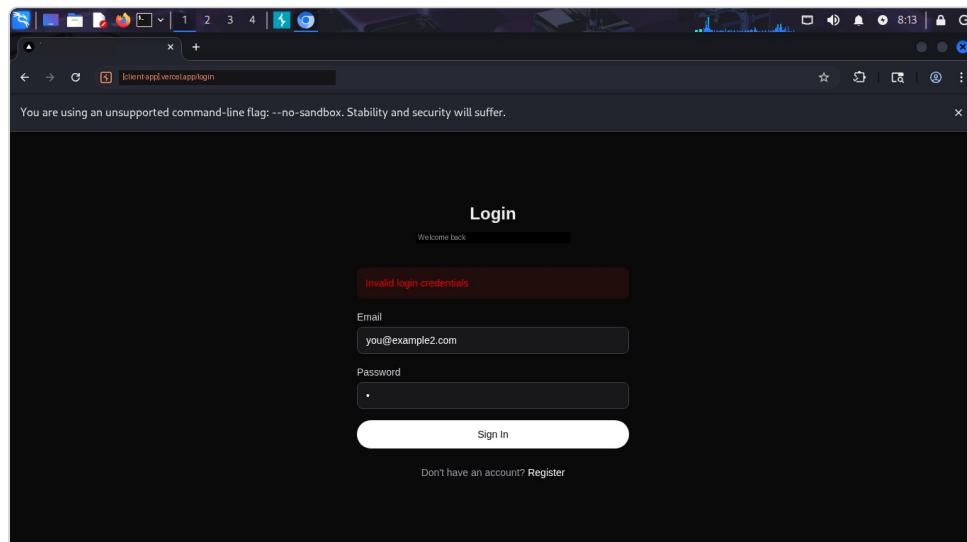


Figure 7. Repeated failed login attempt against the same account returning an unthrottled "Invalid login credentials" response.

IMPACT

In the absence of rate limiting, an attacker can perform high-volume automated credential-stuffing or brute-force attacks against any known account email address. Customers using weak or previously-breached passwords are at meaningfully elevated risk of account takeover.

RECOMMENDATION

- Implement IP-based and/or account-based rate limiting (e.g. lock the account for 15 minutes after 5 consecutive failed attempts).
- Introduce a CAPTCHA or equivalent challenge after 3-5 failed attempts.
- Consider leveraging Supabase Auth's built-in rate-limiting configuration, if available, as a baseline control.

TH-04

Improper Input Validation Leading to Server Crash (HTTP 500)

LOW

CWE: CWE-20 (Improper Input Validation)

OWASP: A10:2025 — Mishandling of Exceptional Conditions

Endpoint: POST /api/orders

DESCRIPTION

As a follow-up to TH-01, submitting a **negative** value for `price` and `total` (e.g. `-10`) does not result in a graceful validation error (HTTP 400); instead, the server raises an unhandled exception and returns `HTTP/2 500 Internal Server Error` with an empty response body (`Content-Length: 0`).

STEPS TO REPRODUCE

- Using the same request from TH-01, set `"price": -10` and `"total": -10`.
- Submit the request.
- Observe the response: `HTTP/2 500 Internal Server Error`, empty body.

EVIDENCE

The screenshot shows the Burp Suite interface with a request and response view. The request is a POST to /api/orders with a JSON body containing negative values for price and total. The response is an HTTP 500 Internal Server Error with an empty body (Content-Length: 0).

Request	Response
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36	1 HTTP/2 500 Internal Server Error
11 Accept: */*	2 Cache-Control: public, max-age=0, must-revalidate
13 Sec-Fetch-Site: same-origin	3 Date: Thu, 18 Jun 2026 15:50:22 GMT
14 Sec-Fetch-Mode: cors	4 Server: Vercel
15 Sec-Fetch-Dest: empty	5 Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
17 Accept-Encoding: gzip, deflate, br	6 X-Matched-Path: /api/orders
18 Priority: u=1,i	7 X-Vercel-Cache: MISS
	8 X-Vercel-Id: cd91::iad1::b9h5g-1781797821563-ad0027576329
	9 Content-Length: 0
	10
	11

Figure 8. Negative-value submission resulting in an unhandled HTTP 500 server error.

IMPACT

While the response body did not leak a stack trace or other sensitive debugging information in this instance, the underlying issue confirms a complete absence of numeric boundary validation, and an unhandled exception path on the order-creation endpoint. Repeated triggering of this condition could be used as a building block for broader denial-of-service or stability-degradation attempts and reflects a general gap in input validation hygiene.

RECOMMENDATION

- Add explicit server-side schema validation (e.g. via Zod) rejecting any non-positive numeric value for price/total/quantity fields with a clean `400 Bad Request` response.
- Wrap order-creation logic in a try/catch block with a generic, non-revealing error response for any unexpected exception.

6.5 Verified Security Controls (Positive Findings)

The following controls were actively tested and confirmed to be functioning correctly. They are documented here to provide a balanced and complete picture of the application's security posture, and to ensure these controls are preserved during future remediation work.

TH-05

Stored XSS Protection — Order Notes Field

PASS

The payload `<img src=x onerror=alert('xss-test-poc')>` was submitted via the order "Notes" field and subsequently rendered within the Admin Panel order view. DOM inspection confirmed the payload was rendered as an inert text node rather than a live HTML element — no script execution occurred. This indicates the front-end (React's default output encoding) correctly escapes user-supplied content before rendering.

EVIDENCE

"" == \$0 (rendered as text node, not parsed as HTML)

Figure 9. DOM inspection confirming the XSS payload was rendered as escaped text, not executed.

TH-06

CSRF Protection on Order Creation Endpoint

PASS

No dedicated CSRF (synchronizer) token was found on `POST /api/orders` ; however, the session cookie is correctly configured with the `SameSite=Lax` attribute, which prevents the cookie from being transmitted on cross-site POST requests. Combined with the endpoint's `Content-Type: application/json` requirement (which cannot be triggered by a simple cross-origin HTML form), this provides effective protection against CSRF on this endpoint.

EVIDENCE



Figure 10. Session cookie attributes confirming SameSite=Lax enforcement.

Administrative Write-Operation Authorization Enforced Server-Side

PASS

While the Admin Panel UI is unauthorizedly reachable (TH-02), the underlying *write* operation that updates an order's status was found to be correctly protected. Attempting to change an order's status via the exposed dropdown, while authenticated as a standard user, resulted in a server-side **403 Forbidden** response, and the UI correctly reverted to the original status.

EVIDENCE

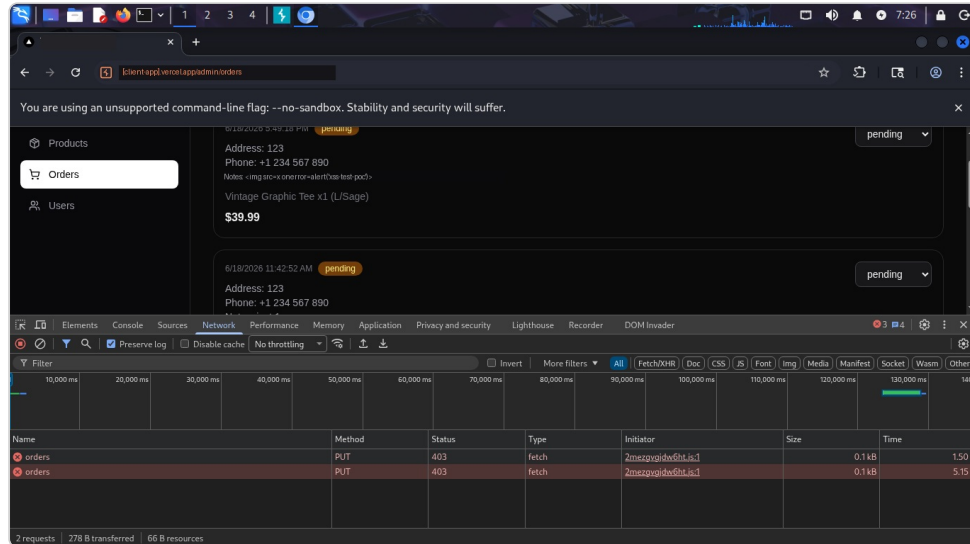


Figure 11. PUT request to update order status returning HTTP 403 Forbidden for a non-privileged account.

This finding directly informs the remediation strategy for TH-02: the necessary authorization-check logic already exists in the codebase for write operations and should be extended to cover the equivalent read operations.

An attempt was made to retrieve a first test account's order directly by ID (`GET /api/orders/{\"order_id\"}`) while authenticated as a second, independent test account, to assess for Insecure Direct Object Reference (IDOR). The endpoint returned `404 Not Found`, indicating that direct per-ID retrieval is not a supported access pattern for standard users — order data is instead scoped exclusively to the requesting user's own session via a separate, session-bound endpoint. No IDOR was demonstrated through this vector.

EVIDENCE

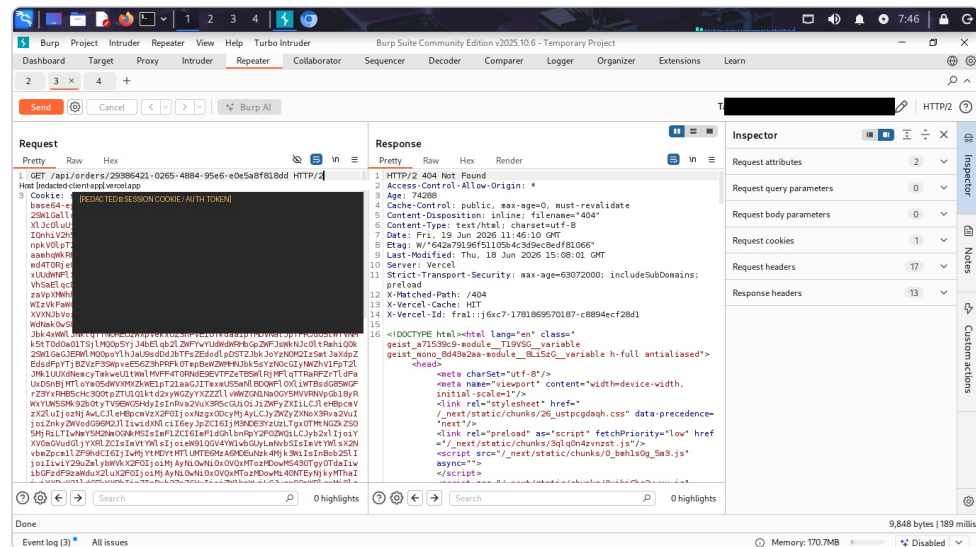


Figure 12. Cross-account direct order ID access attempt returning 404 Not Found. (Session cookie redacted from this capture.)

7. Strategic Recommendations

Beyond the specific remediation steps listed under each finding, the following strategic observations are offered to help the development team address the root cause of the issues identified in this assessment:

7.1 Establish a Consistent Server-Side Trust Boundary

The most consequential findings in this report (TH-01, TH-02) share a common root cause: a tendency to trust data or routing decisions made on the client side. The application correctly enforces authorization on *write* operations (TH-07) but not on *read* operations (TH-02), and correctly renders product prices for display but does not re-derive them server-side at the point of order creation (TH-01). The development team should adopt a blanket rule: **any request that reads or writes data must independently verify both the requester's identity/role and the integrity of any business-critical value (price, quantity, status) on the server, regardless of what the client UI displays or submits.**

7.2 Audit All Routes Generated Alongside the Admin Panel

Given that the write-protection logic in TH-07 already exists and functions correctly, it is highly likely that extending the same authorization check to the corresponding GET/read handlers for `/admin/orders` and `/admin/products` will resolve TH-02 with minimal engineering effort. This should be treated as the highest-priority fix arising from this assessment.

7.3 Strengthen Authentication Hardening

Implementing rate limiting (TH-03) alongside the existing authentication flow will substantially reduce the platform's exposure to automated credential attacks, particularly important given the PII exposure identified in TH-02.

7.4 Adopt Schema-Based Input Validation

A consistent, centrally-defined validation schema (e.g. Zod or equivalent) applied to all API route handlers would simultaneously address TH-01 and TH-04, and reduce the likelihood of similar issues being introduced in future feature development.

8. Conclusion

This assessment identified a small number of high-impact vulnerabilities concentrated around two specific trust-boundary gaps — order pricing and administrative panel authorization — rather than a broad pattern of weak security practice across the application. This is a meaningful and encouraging signal: the controls that *are* in place (CSRF protection, XSS output encoding, write-operation authorization, session cookie hardening) are implemented correctly and consistently, suggesting the remaining gaps are addressable with focused, well-scoped engineering effort rather than a fundamental architectural overhaul.

It is the Assessor's professional opinion that **Findings TH-01 and TH-02 should be treated as release blockers** for any production or public launch of the Client App platform, given their direct financial and customer-privacy implications. Findings TH-03 and TH-04 are recommended for remediation in the near term as part of standard security hardening.

The Assessor would welcome the opportunity to perform a follow-up retest once remediation work is complete, to verify that the identified issues have been fully resolved and that no regressions have been introduced.

Thank you. This concludes the technical findings of this engagement. For any questions regarding this report or its findings, please contact the Security Assessor directly.

Appendix A — Disclaimer & Confidentiality

This report reflects the security posture of the Client App web application (<https://redacted-client-app.vercel.app>) as observed during the engagement window of June 18-19, 2026. It is not a guarantee of the absence of vulnerabilities beyond those described herein; a penetration test reflects a point-in-time, best-effort assessment within the agreed scope and time constraints, and should not be treated as a substitute for ongoing secure development practices, code review, or continuous security monitoring.

All testing described in this report was performed under explicit, signed written authorization obtained from the application owner prior to the commencement of any testing activity, in accordance with the engagement's Rules of Engagement. No production customer data was modified, deleted, or exfiltrated during this engagement; all order and account data referenced in this report's evidence was created by the Assessor specifically for testing purposes.

This report and its contents are classified as **Confidential** and are intended solely for the use of the Client (Client App application owner) and the Security Assessor. It should not be redistributed, published, or disclosed to any third party without the express written consent of both parties, with the exception of the Assessor's right to reference the existence and general nature of this engagement (without disclosing vulnerability-specific technical detail or any client-identifying information) for professional portfolio purposes.

Document Control	
Report Classification	Confidential
Report Version	1.0
Prepared By	Shadi Ibrahim — Penetration Tester / Security Consultant
Date Issued	June 19, 2026